



Παράλληλος και Κατανεμημένος Υπολογισμός I

Παράλληλος υπολογισμός

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Παράλληλα συστήματα

Παράλληλος υπολογισμός 1

- Έχει στόχο την βελτίωση των επιδόσεων ενός υπολογισμού: να μειώσει τον χρόνο εκτέλεσης ή / και να αυξήσει το μέγεθος των δεδομένων του υπολογισμού.
- Προϋποθέτει τη χρήση υπολογιστικού συστήματος που υποστηρίζει πραγματικά ταυτόχρονη εκτέλεση, όχι απλά ταυτοχρονισμό σε έναν επεξεργαστή.
Συνήθως:
- Πολλαπλοί επεξεργαστές (πχ multicores)
- Πολλαπλοί δικτυωμένοι υπολογιστές (πχ clusters)
- Πολλαπλές ALUs (πχ gru's -- περίπου)

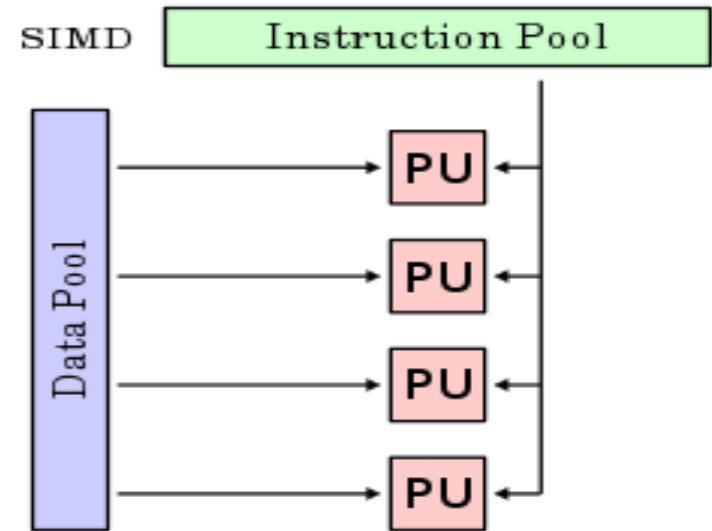
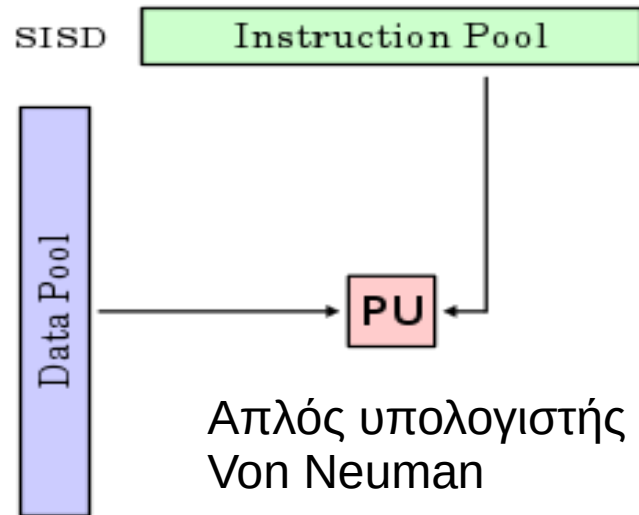
Παράλληλος υπολογισμός 2

- Ξεκινά από έναν ακολουθιακό αλγόριθμο (ή πρόγραμμα) που λύνει ένα πρόβλημα και το αναλύει σε επιμέρους παράλληλους υπολογισμούς για να βελτιώσει τις επιδόσεις του.
- Επομένως σχεδιάζεται ώστε να διαθέτει πολλαπλές ταυτόχρονες ροές ελέγχου.
- Δεν υποφέρει από εγγενή μη-αιτιοκρατική εκτέλεση, τυχαίες καθυστερήσεις και εξωτερικά γεγονότα, αλλά κυρίως από σφάλματα σχεδίασης ή υλοποίησης.

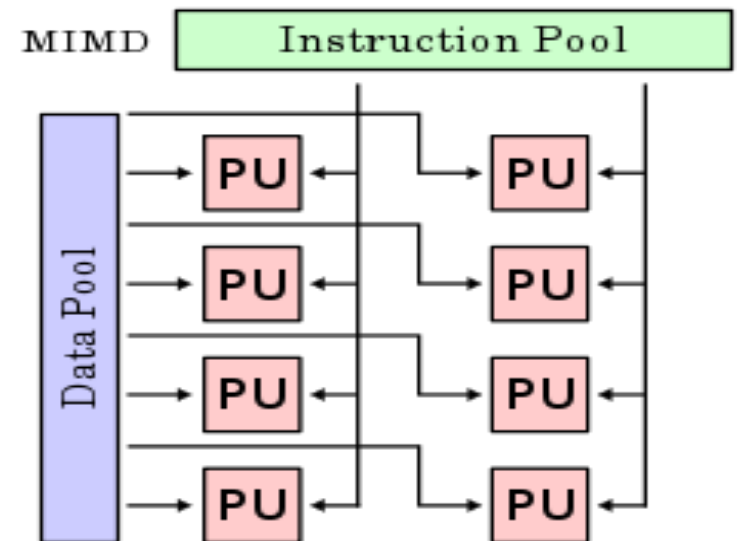
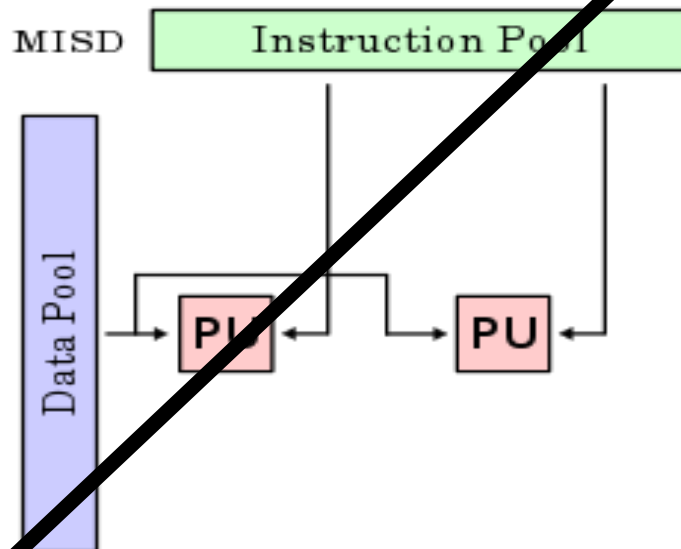
Μοντέλα παράλληλων συστημάτων 1

- Πόσες εντολές εκτελούνται σε κάθε χρονική στιγμή;
Μια (Single) ή πολλές (Multiple) Εντολές (Instructions);
- Πόσα δεδομένα επεξεργαζόμαστε σε κάθε χρονική στιγμή;
Ένα (Single) ή Πολλά (Multiple) Δεδομένα (Data);

Μοντέλα παράλληλων συστημάτων Flynn (1964)



Συστήματα Παραλληλισμού Δεδομένων

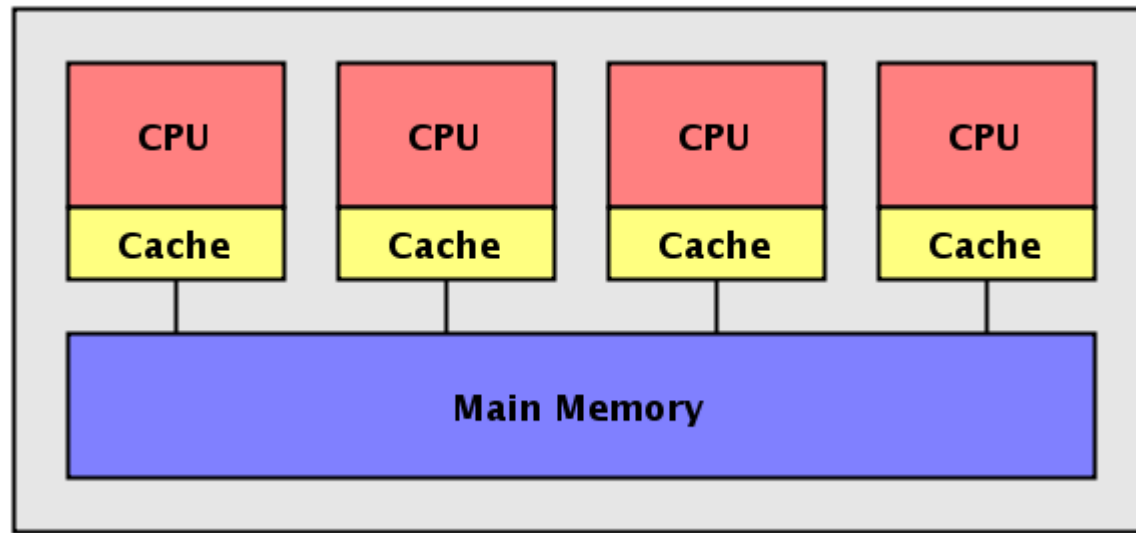


Συστήματα Μοιραζόμενης ή Κατανεμημένης Μνήμης

Μοντέλα παράλληλων συστημάτων 2

- MIMD μοιραζόμενης μνήμης (πχ multicores)
- MIMD κατανεμημένης μνήμης (πχ clusters)
- SIMD : ανυσματικές ALUs σε κάθε CPU.
- (περίπου) SIMD: επιταχυντές (πχ GPUs)

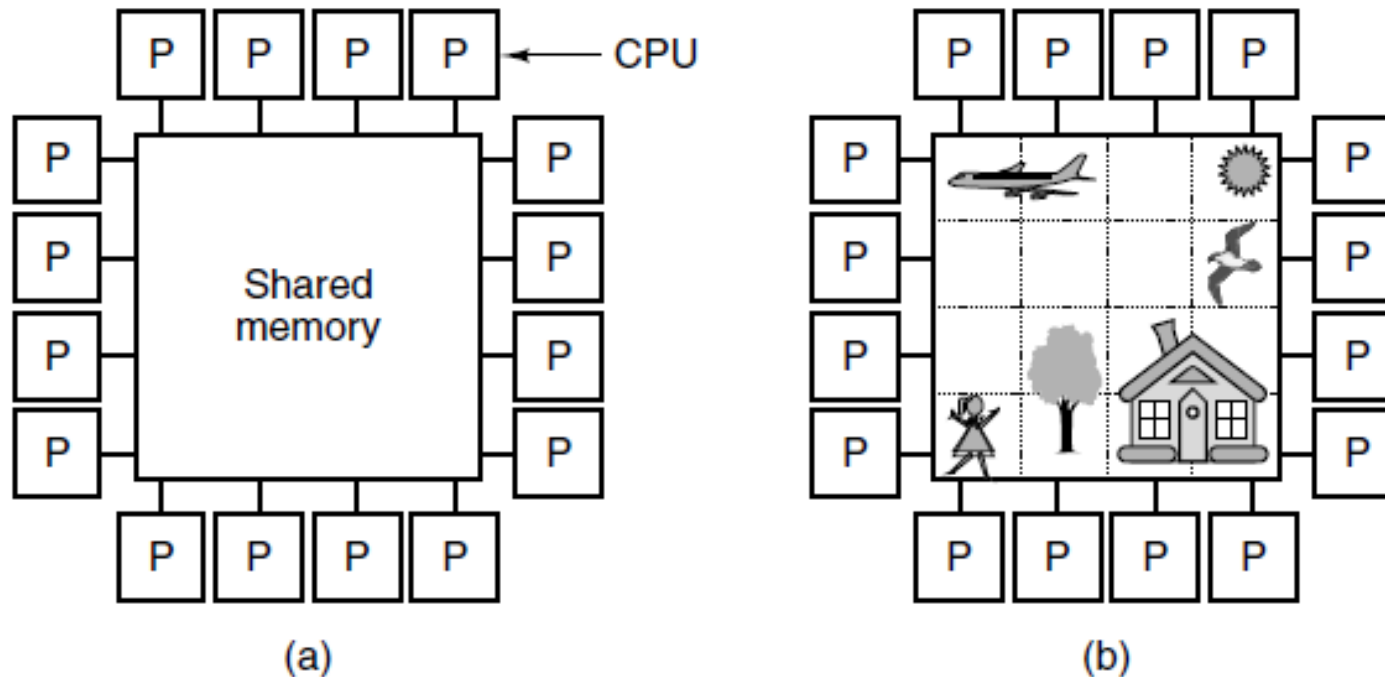
Σύστημα μοιραζόμενης μνήμης 1



Τυπική υλοποίηση: υπολογιστές με πολυπύρηνες CPUs.

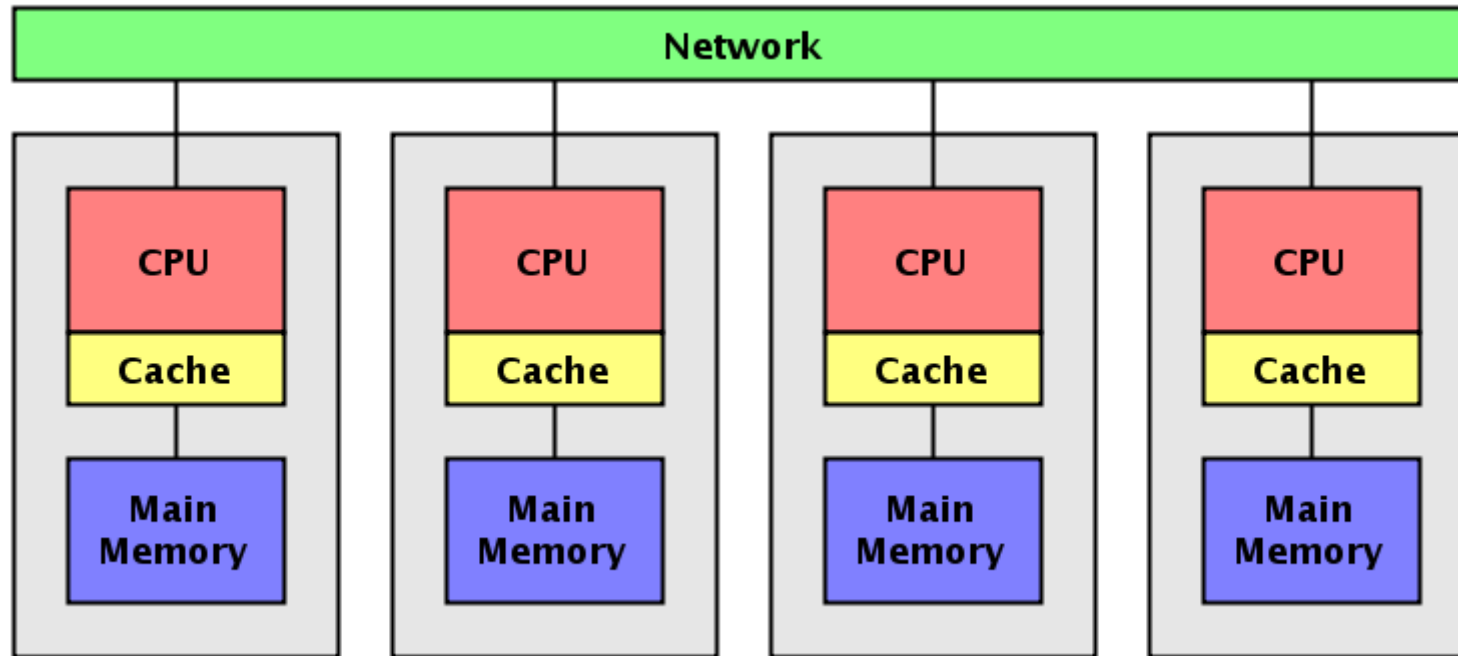
Υπέρ: εύκολος προγραμματισμός (μοιραζόμενες μεταβλητές).
Κατά: δύσκολη επέκταση (συνήθως απαιτείται νέο υλικό)

Σύστημα μοιραζόμενης μνήμης 2



Τυπική χρήση παράλληλου υπολογιστικού συστήματος μοιραζόμενης μνήμης: μια εικόνα επιμερίζεται λογικά σε 16 τμήματα, και κάθε επεξεργαστής εκτελεί ένα νήμα που επεξεργάζεται ένα τμήμα.

Σύστημα κατακευμεμένης μνήμης 1

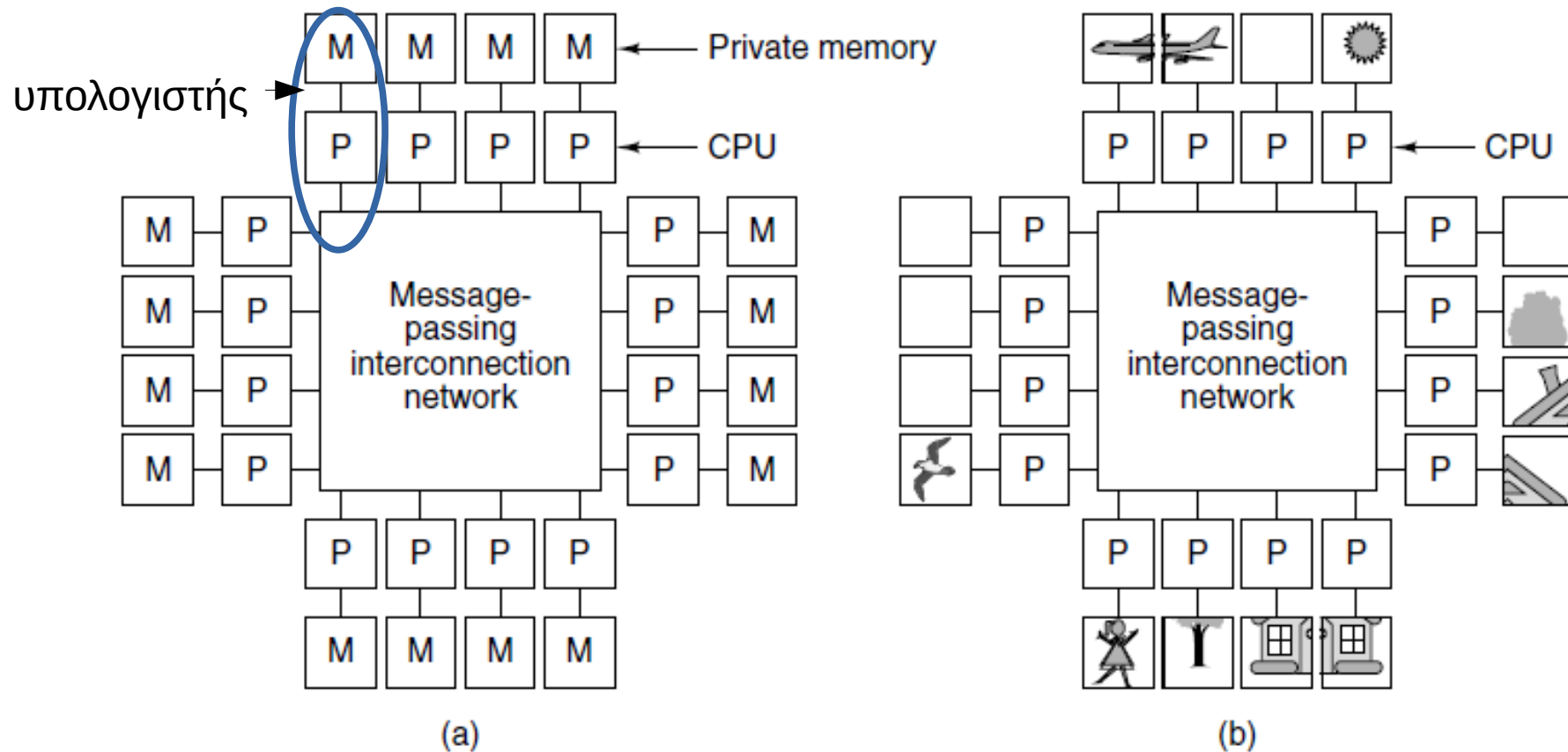


Τυπική υλοποίηση: συστοιχία (cluster) υπολογιστών, κέντρα δεδομένων (data centers).

Υπέρ: εύκολη επέκταση (αγορά έτοιμου εξοπλισμού)

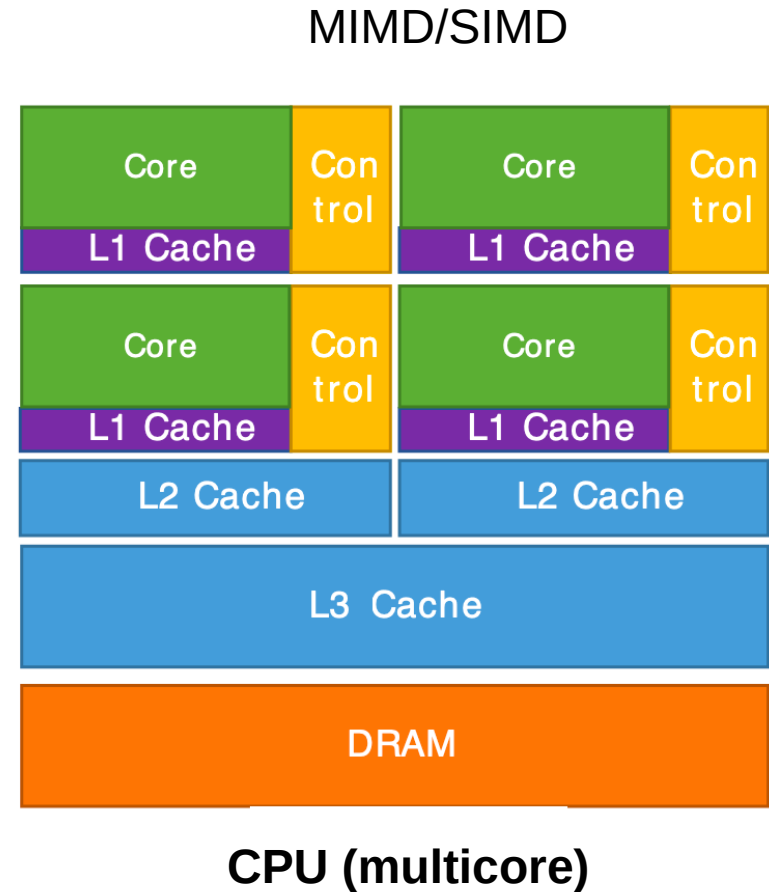
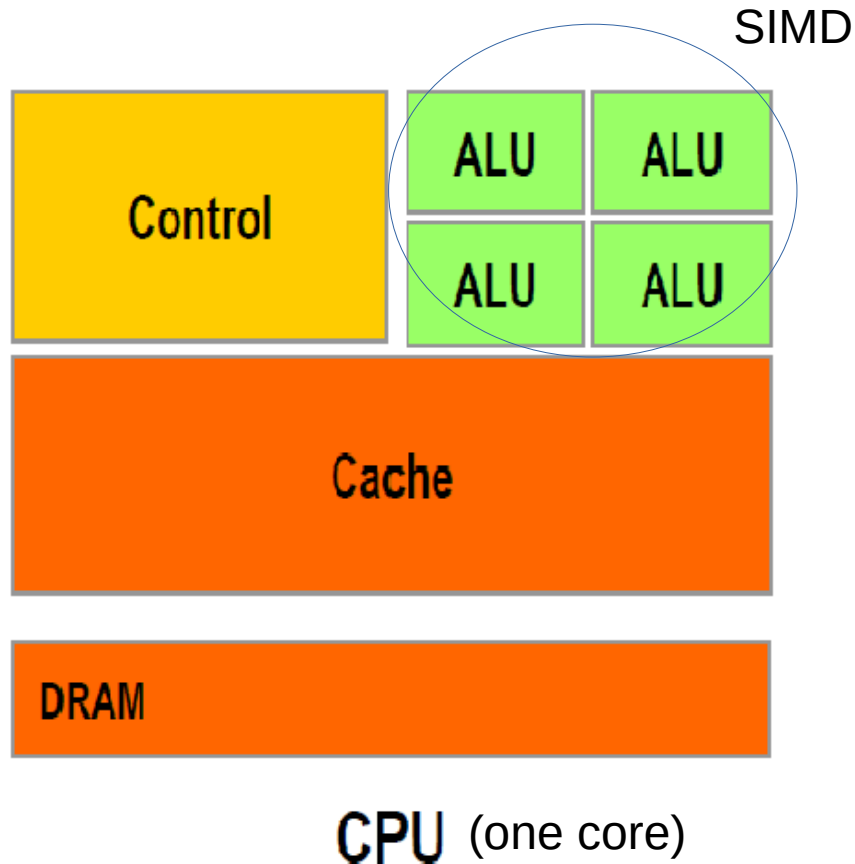
Κατά: δυσκολότερος προγραμματισμός (μεταβίβαση μηνυμάτων).

Σύστημα κατανεμημένης μνήμης 2



Τυπική χρήση παράλληλου υπολογιστικού συστήματος κατανεμημένης μνήμης: μια εικόνα επιμερίζεται φυσικά σε 16 τμήματα, και κάθε υπολογιστής εκτελεί μια διεργασία που επεξεργάζεται ένα τμήμα στην ιδιωτική του μνήμη.

Επιταχυντές 1

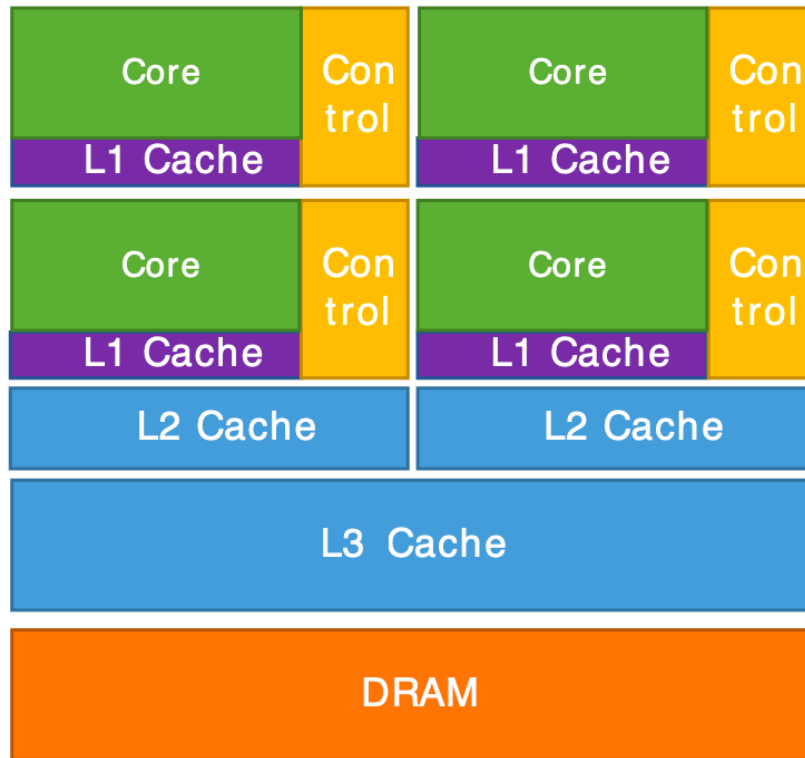


Πολλές ALUs σε μια CPU: χρήση εντολών SIMD (πχ Intel AVX).

Πολλές CPUs (cores) σε ένα multicore chip: MIMD/SIMD.

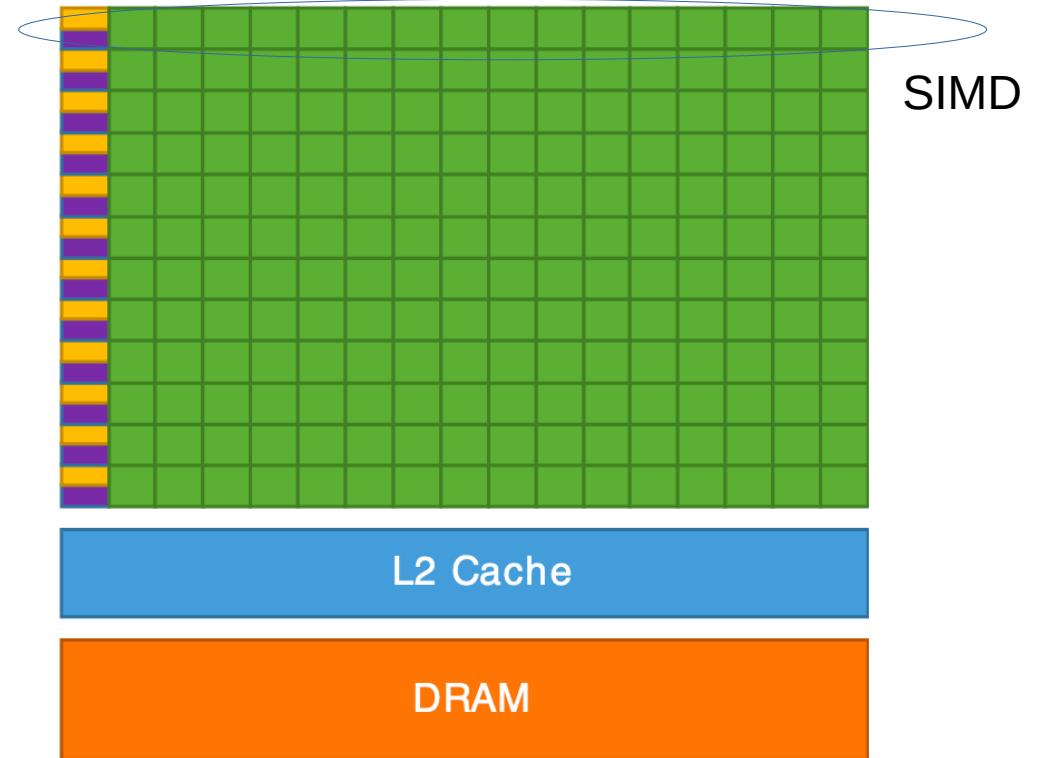
Επιταχυντές 2

MIMD/SIMD



CPU

MIMD/SIMD



GPU

Τυπική υλοποίηση: μονάδες επεξεργασίας γραφικών (GPUs)

Υπέρ: πολύ υψηλές επιδόσεις

Κατά: δυσκολότερος προγραμματισμός (γνώση αρχιτεκτονικής)
περιορισμένο (αλλά χρήσιμο) σύνολο εφαρμογών (for loops)

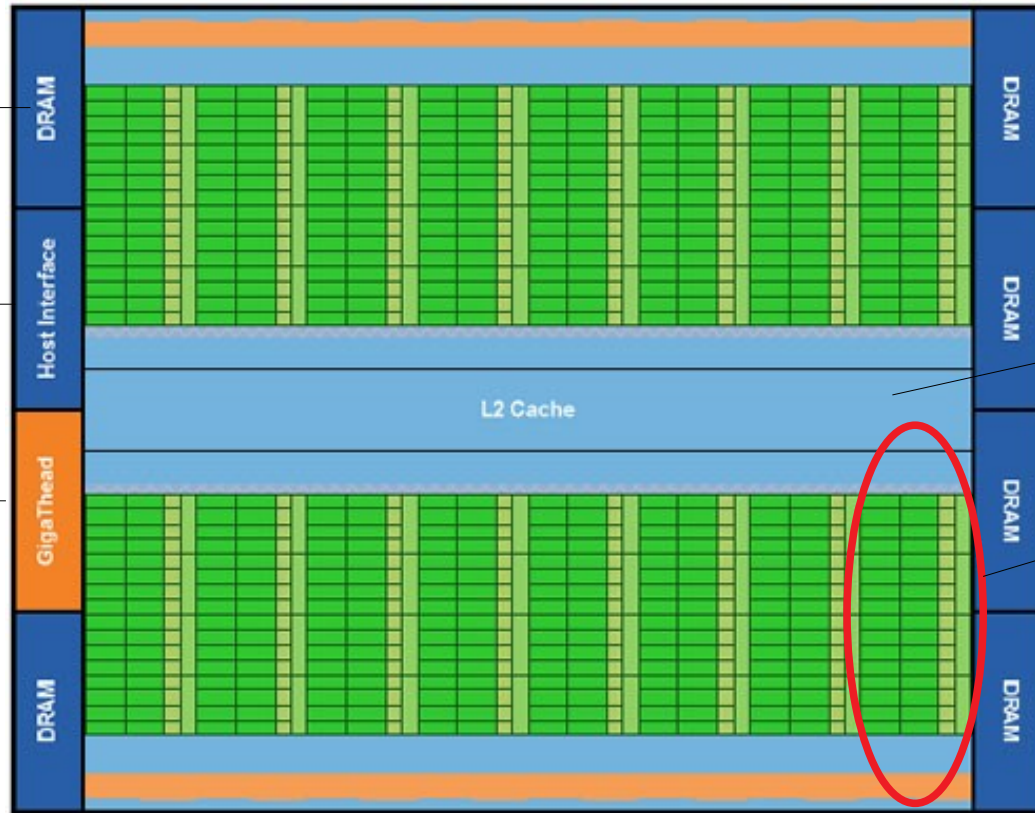
Επιταχυντές 2

Μοιραζόμενη μνήμη
Μόνο ανάγνωσης
μεταξύ ανυσματικών
επεξεργαστών

Επικοινωνία με
CPU και μνήμη

Δρομολόγηση
νημάτων σε
επεξεργαστές

Συνολικά 16
ανυσματικοί
επεξεργαστές



Ιδιωτική μνήμη
ανάγνωσης εγγραφής
για κάθε ανυσματικό
επεξεργαστή
μοιραζόμενη μόνο
μεταξύ των ALUs του

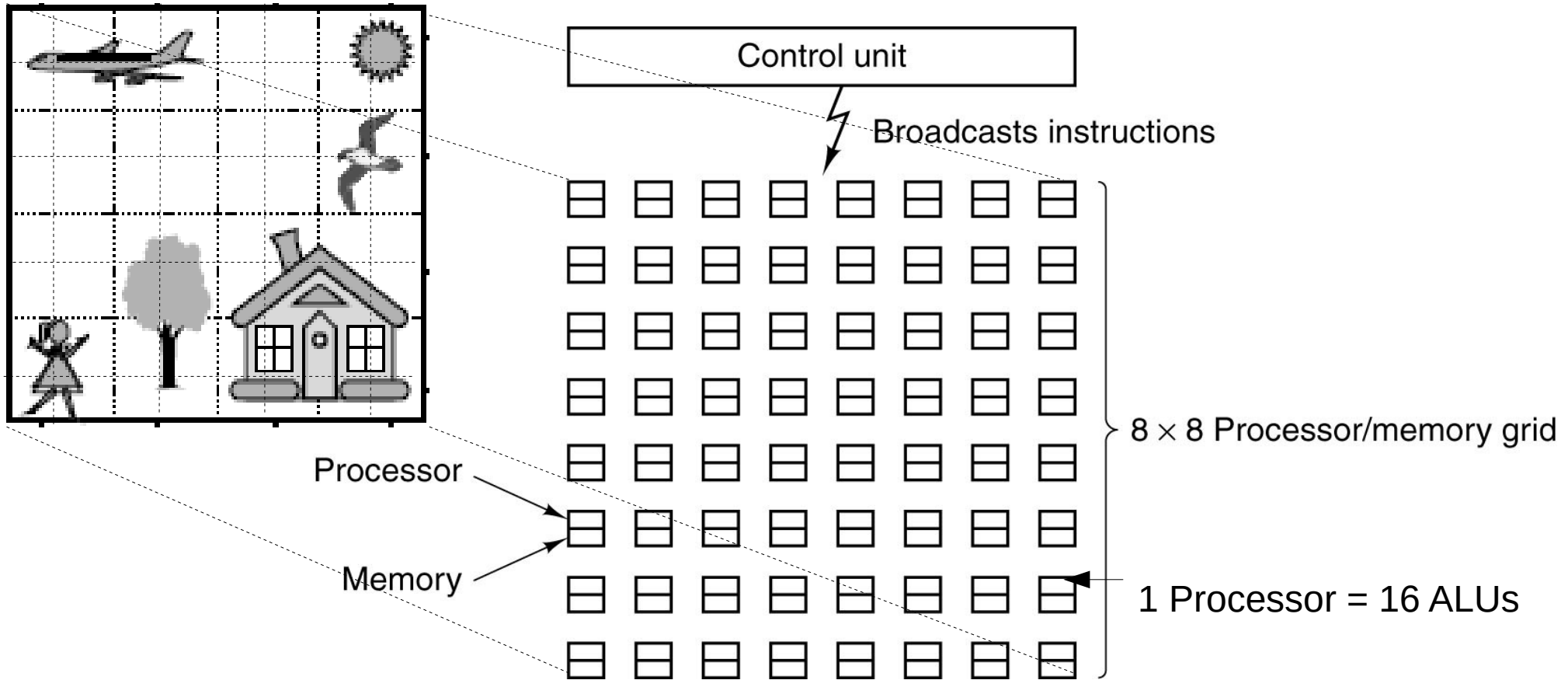
Ανυσματικός
Επεξεργαστής:
Εκτελεί ροές ελέγχου
(νήματα)
1 CU ελέγχει
32 ALUs που
εκτελούν την ίδια
εντολή σε διαφορετικά
δεδομένα η κάθε μια.

Τυπική υλοποίηση: μονάδες επεξεργασίας γραφικών

Σε πολλές σύγχρονες CPUs υπάρχουν ενσωματωμένα συστήματα
παραλληλισμού δεδομένων.

Υπέρ: επιδόσεις Κατά: όχι γενικές εφαρμογές (επιταχυντής).

Επιταχυντές 3



Τυπική χρήση παράλληλου υπολογιστικού συστήματος παραλληλισμού δεδομένων: μια εικόνα επιμερίζεται σε 64 τμήματα, και κάθε ένας από τους 64 επεξεργαστές αναθέτει την επεξεργασία του τμήματος σε 16 εσωτερικές ALUs (SIMD).

Αρχιτεκτονικές παράλληλων υπολογιστικών συστημάτων 1

- Τρόπος οργάνωσης και πρόσβασης στα συστήματα μοιραζόμενης μνήμης:

Uniform Memory Access (UMA, ενιαία μνήμη) ή

Non Uniform Memory Access (NUMA, πχ μνήμη ανά core / chip και διασύνδεση on-chip / on-board)

- Τρόπος σύνδεσης των υπολογιστών στα συστήματα κατανεμημένης μνήμης:

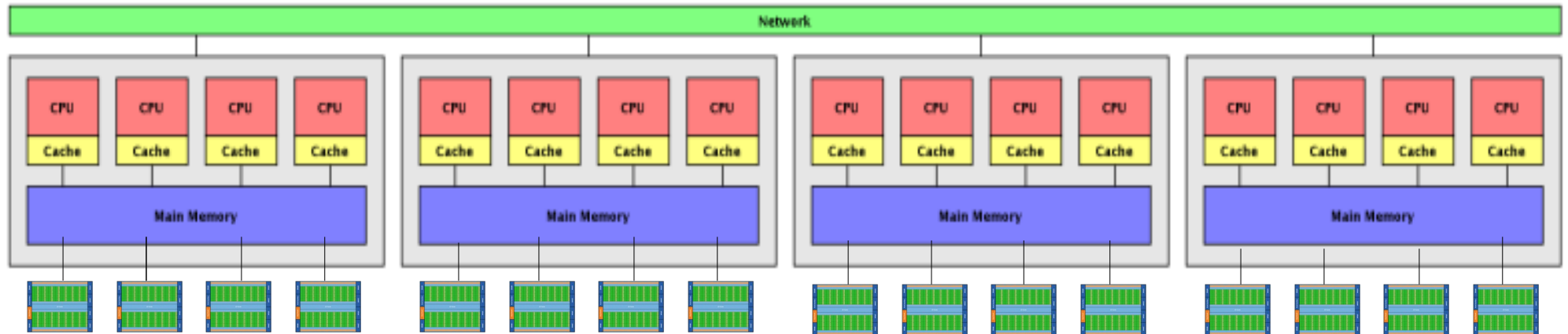
τοπικό δίκτυο (πχ Gigabit Ethernet) ή

ειδική διασυνδετική υποδομή (πχ Infiniband).

Αρχιτεκτονικές παράλληλων υπολογιστικών συστημάτων 2

- Systems on Chip (SoCs): επεξεργαστές που περιέχουν
CPUs διαφόρων τύπων (ελαφρές και ισχυρές μονάδες CPUs) ή
CPUs και GPUs (πχ επεξεργαστές M1, mic, ...)
- Επεξεργαστές ειδικού σκοπού:
Field Programmable Gate Arrays (FPGAs) πχ για TensorFlow, Bitcoin mining

Αρχιτεκτονικές παράλληλων υπολογιστικών συστημάτων 2



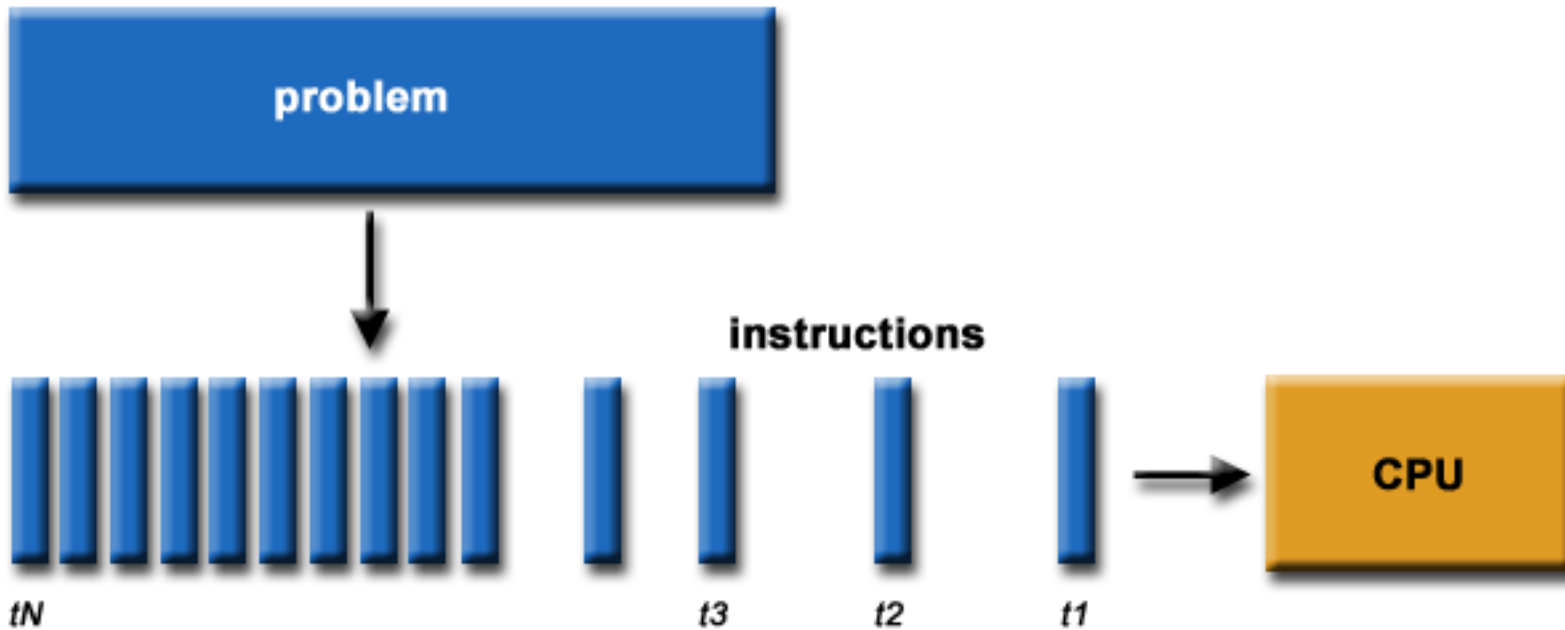
Ετερογενής παραλληλισμός

- Στις σύγχρονες συστοιχίες (clusters) και κέντρα δεδομένων (data centers) κάθε κόμβος μπορεί να έχει πολλούς επεξεργαστές και επιταχυντές γραφικών (όχι απαραίτητα ίσο αριθμό).



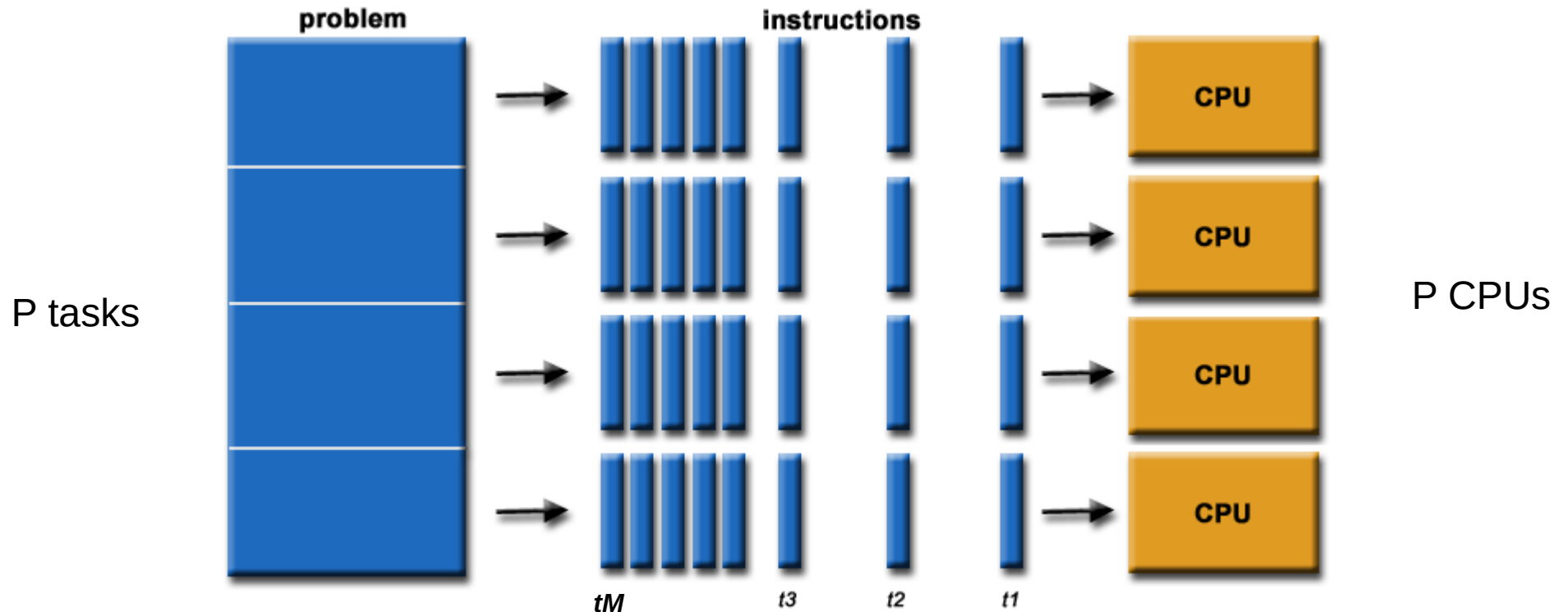
Παράλληλος προγραμματισμός

Ακολουθιακός υπολογισμός



Ένα ακολουθιακό πρόγραμμα εκτελεί N εντολές για την επίλυση ενός προβλήματος βάσει ενός αλγορίθμου.

Παράλληλος υπολογισμός



Ένα παράλληλο πρόγραμμα εκτελεί ταυτόχρονα P εργασίες σε P CPUs (με $M < N$ εντολές ανά εργασία - task) για την επίλυση του ίδιου προβλήματος βάσει του αντίστοιχου παράλληλου αλγορίθμου.

Παράλληλος vs Ταυτόχρονος Υπολογισμός 1

- Στον ταυτόχρονο υπολογισμό ο ταυτοχρονισμός **επιβάλλεται** από την φύση του προβλήματος και είναι **σχετικά τυχαίος**, δεν μπορούμε να καθορίσουμε πότε θα συμβεί.
- Πχ: Σε μια εφαρμογή διαχείρισης τραπεζικών λογαριασμών ο ταυτοχρονισμός είναι απαραίτητος αλλιώς θα είχαμε σοβαρή πτώση του χρόνου απόκρισης.
- Δεν μπορούμε να γνωρίζουμε πότε θα γίνουν ταυτόχρονες συναλλαγές σε ένα λογαριασμό, σχεδιάζουμε για την χειρότερη περίπτωση.

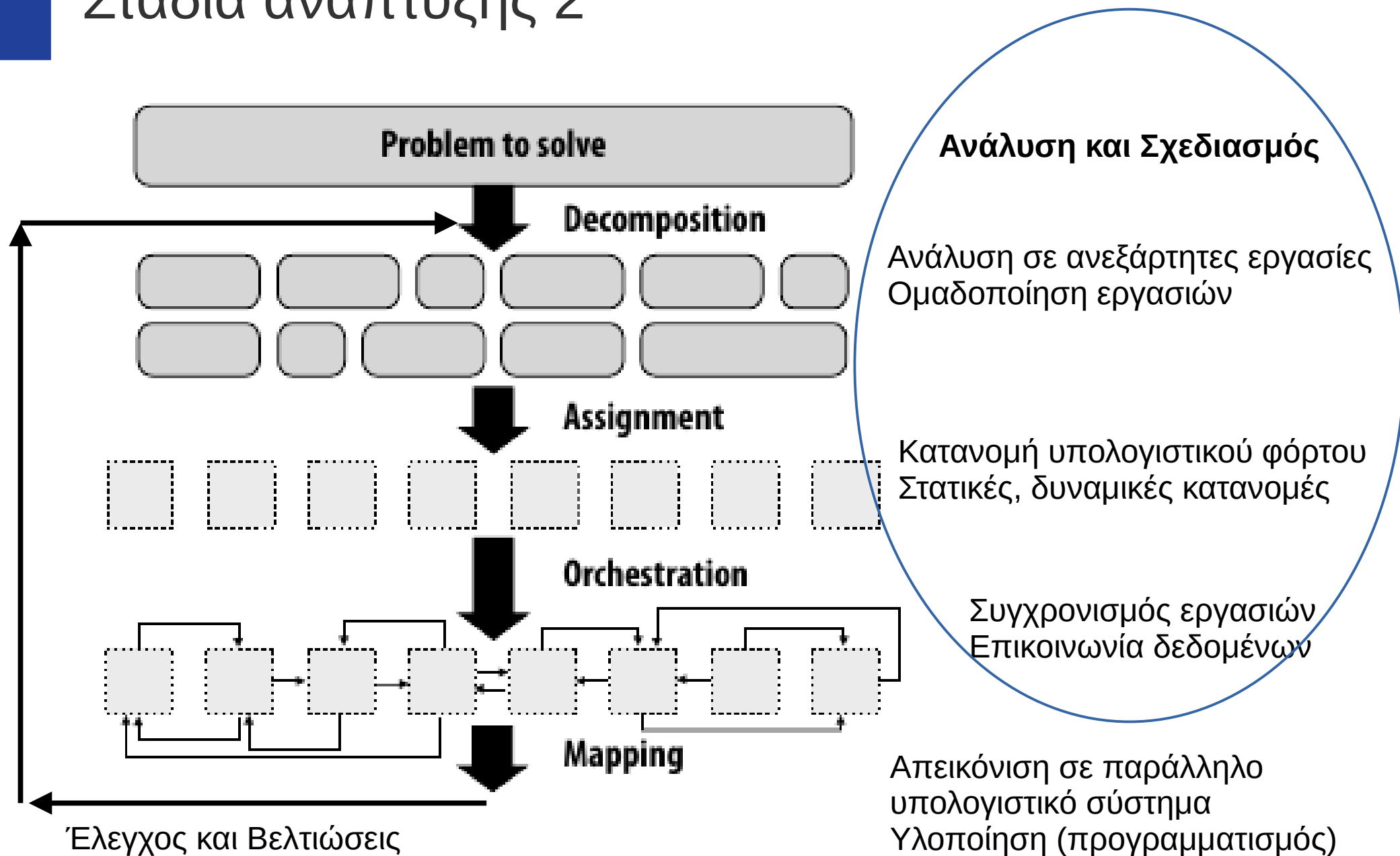
Παράλληλος vs Ταυτόχρονος Υπολογισμός 2

- Στον παράλληλο υπολογισμό ο ταυτοχρονισμός **εισάγεται** στον σχεδιασμό της εφαρμογής για να επιτευχθεί καλύτερος χρόνος εκτέλεσης, και **δεν είναι τυχαίος**, εμείς καθορίζουμε πότε και πώς θα συμβεί.
- Πχ: Σε μια εφαρμογή επεξεργασίας (rendering) μιας ψηφιακής ταινίας ο παραλληλισμός εισάγεται γιατί η ακολουθιακή εκτέλεση είναι πολύ αργή.
- Γνωρίζουμε ακριβώς πότε και πως θα γίνει πχ ο διαμοιρασμός των τμημάτων της ταινίας, και η τελική σύνθεση του αποτελέσματος.

Στάδια ανάπτυξης παράλληλου προγράμματος

- Ανάλυση και Σχεδιασμός
- Υλοποίηση
- Έλεγχος και Βελτιώσεις

Στάδια ανάπτυξης 2



Ανάλυση και Σχεδιασμός

- Ποια τμήματα του αλγορίθμου μπορούν να εκτελεστούν ανεξάρτητα μεταξύ τους; Σε ποια τμήματα υπάρχουν εξαρτήσεις;
- Πιθανή αλλαγή / τροποποίηση αλγορίθμου.
- Μέγεθος, ομαδοποίηση, εργασιών: πολλές μικρές ή λίγες μεγάλες εργασίες;
- Κατανομή υπολογιστικού φόρτου.
- Απαιτήσεις για συγχρονισμό εργασιών και επικοινωνία δεδομένων μεταξύ των εργασιών.

Υλοποίηση

- Μοντέλο υπολογιστικού συστήματος που θα εκτελεστεί το πρόγραμμα.
- Γλώσσα προγραμματισμού, δομές δεδομένων.
- Χρήση προγραμματιστικών προτύπων και παράλληλων βιβλιοθηκών. Λύση όσο το δυνατό σε υψηλό επίπεδο, χρήση δομών χαμηλού επιπέδου μόνο αν είναι απολύτως απαραίτητο.
- Εργασία (Task): διεργασία, νήμα ... γενικά ρουτίνα που μπορεί να εκτελεστεί ανεξάρτητα από το λειτουργικό σύστημα ή το σύστημα χρόνου εκτέλεσης.

Έλεγχος

- Μέτρηση χρόνων ακολουθιακής και παράλληλης εκτέλεσης, εκτίμηση επιδόσεων.
- Διερεύνηση για πιθανά σημεία συμφόρησης για βελτίωση της επίδοσης.
- Μελέτη κλιμάκωσης της εφαρμογής μας (μέγεθος προβλήματος, πλήθος επεξεργαστών).

Βασική μετρική: Επιτάχυνση (Speedup)*

Επιτάχυνση =

Χρόνος εκτέλεσης του ακολουθιακού προγράμματος για μέγεθος προβλήματος N : $T(N, 1)$

Χρόνος εκτέλεσης παράλληλου προγράμματος για μέγεθος προβλήματος N σε P επεξεργαστές: $T(N, P)$

$$\text{Speedup } S(N, P) = T(N, 1) / T(N, P)$$

$$T(N, P) \leq T(N, 1) / P \text{ δηλαδή } S(N, P) \leq P$$

* υπάρχουν αρκετές άλλες μετρικές (συζητούνται αργότερα)

Βελτιώσεις

- Αλγοριθμικές βελτιώσεις (πχ αλγόριθμοι εγγενώς παράλληλοι ή ακολουθιακοί, μείωση εξαρτήσεων).
- Βελτιώσεις στην επικοινωνία (μείωση απαιτήσεων συγχρονισμού και επικοινωνίας).
- Βελτιώσεις που σχετίζονται με την γλώσσα και το πρόγραμμα (πχ δομές δεδομένων, μηχανισμοί ταυτοχρονισμού).
- Βελτιώσεις που σχετίζονται με το υπολογιστικό σύστημα (πχ κρυφή μνήμη, μέγεθος και πλήθος εργασιών).

Μερικοί απλοί κανόνες 1

- Εστιάζουμε στα υπολογιστικά εντατικά (CPU-bound) τμήματα του προγράμματος με βάση την πολυπλοκότητα του ακολουθιακού αλγορίθμου. (Μπορούμε να εκτελέσουμε profiling).
- Συνήθως εστιάζουμε σε απλούς ή πολλαπλούς (ένθετους) βρόχους επανάληψης for loops (ή αναδρομικές κλήσεις).
- Συνήθως δεν ασχολούμαστε με βρόχους αρχικοποιήσεων ή τελικής εμφάνισης αποτελεσμάτων.
- Συνήθως αποφεύγουμε τμήματα κώδικα που περιλαμβάνουν I/O (I/O-bound), εκτός αν έχουμε ειδικό λόγο ή κατάλληλο hardware.

Μερικοί απλοί κανόνες 2

- Ελέγχουμε μήπως κάποιος άλλος αλγόριθμος δίνει καλύτερο παραλληλισμό, δηλαδή έχει λιγότερες εξαρτήσεις
- Σχεδιάζουμε από το υψηλότερο επίπεδο παραλληλισμού (αδρομερές) προς το χαμηλότερο (λεπτομερές)
- Σχεδιάζουμε παραμετρικά ως προς τον αριθμό των παράλληλων εργασιών / επεξεργαστών (κλιμάκωση)
- Εφαρμόζουμε αλγοριθμικά και προγραμματιστικά πρότυπα όπου είναι εφικτό
- Αν η γλώσσα μας έχει έτοιμες παράλληλες βιβλιοθήκες τις χρησιμοποιούμε



Δύο προσεγγίσεις στην ανάλυση

Δύο προσεγγίσεις στην ανάλυση

- Παραλληλισμός Δεδομένων

Ο παραλληλισμός βασίζεται στις δομές δεδομένων ή/και στον αλγόριθμο που τις επεξεργάζεται

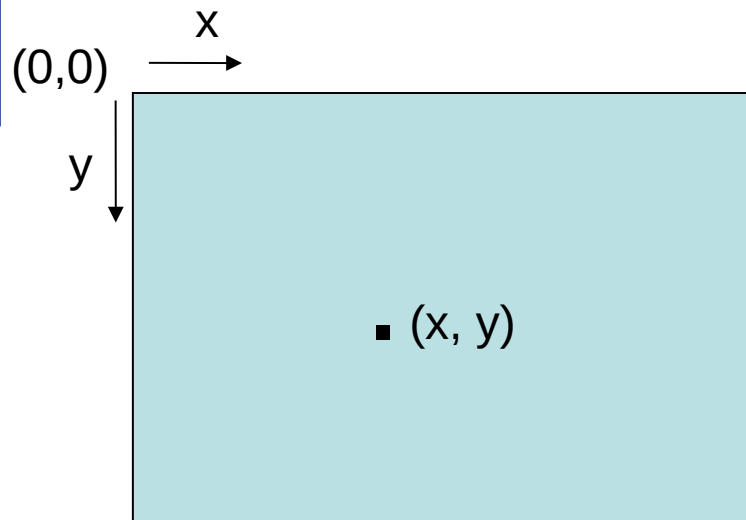
- Παραλληλισμός Εργασιών

Ο παραλληλισμός βασίζεται στο γράφημα κλήσεων συναρτήσεων / μεθόδων του αλγορίθμου / προγράμματος

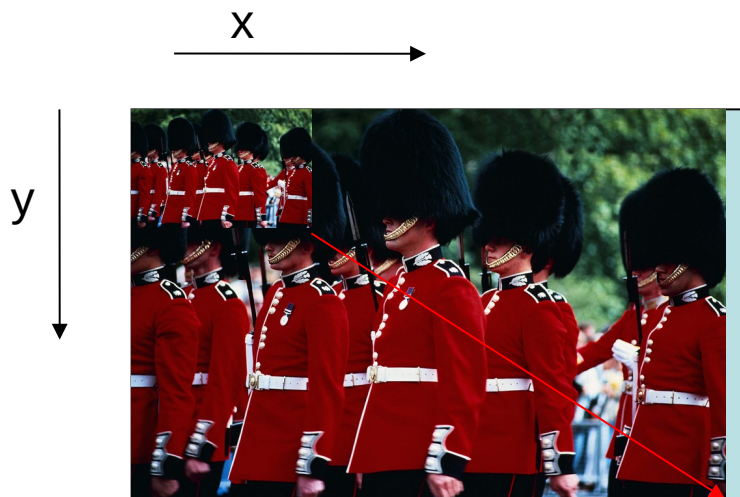
Παραλληλισμός δεδομένων 1

- Ανάλυση με βάση την οργάνωση (ή τις δομές) δεδομένων που χρησιμοποιεί ο ακολουθιακός αλγόριθμος.
- Έχει εφαρμογή σε ένα πολύ σημαντικό υποσύνολο υπολογιστικά εντατικών υπολογισμών ευρείας χρήσης.
- Προσφέρει εύκολες και αποδοτικές λύσεις, με δυνατότητες αυτοματοποίησης των προτύπων και απεικόνισης λύσεων σε γενικούς αλλά και ειδικούς επεξεργαστές.

Εικόνες: απλές λειτουργίες



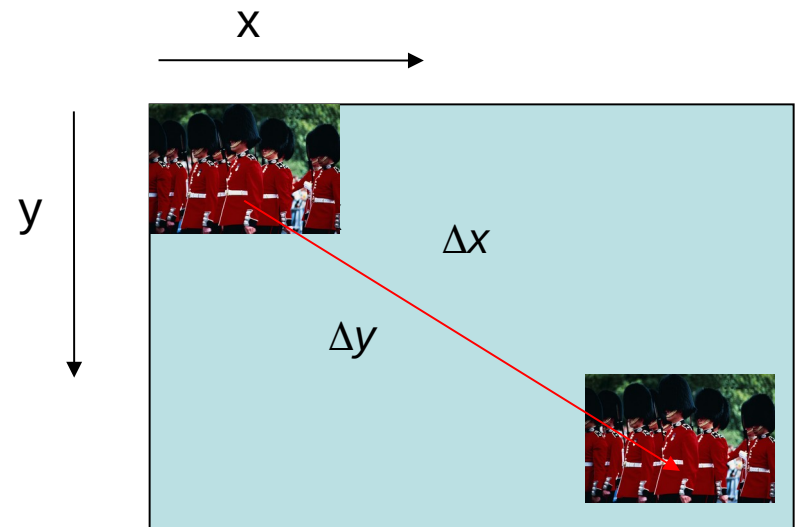
Επανάληψη για κάθε x, y : $O(n^2)$



Κλιμάκωση

$$x' = x * S_x$$

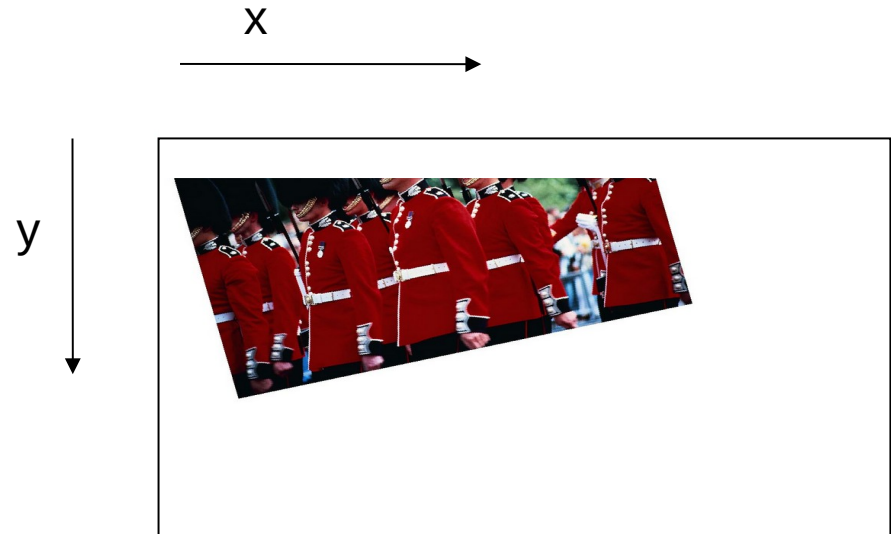
$$y' = y * S_y$$



Μετατόπιση

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$



Περιστροφή

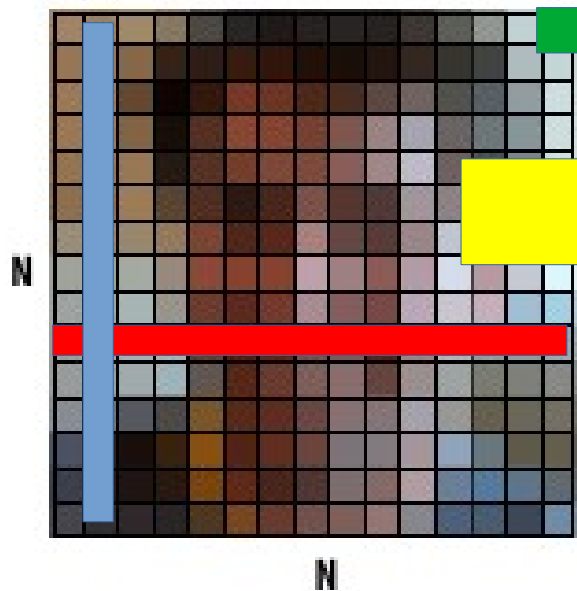
$$x' = x \cos\theta + y \sin\theta$$

$$y' = -x \sin\theta + y \cos\theta$$

Παραλληλισμός δεδομένων 2

https://en.wikipedia.org/wiki/Category:Computer_graphics_algorithms

https://en.wikipedia.org/wiki/Digital_image_processing



- Οι εργασίες είναι ανεξάρτητες μεταξύ τους.
- Επεξεργασία ενός πίνακα δύο διαστάσεων ανά στοιχείο, γραμμή, στήλες ή blocks.

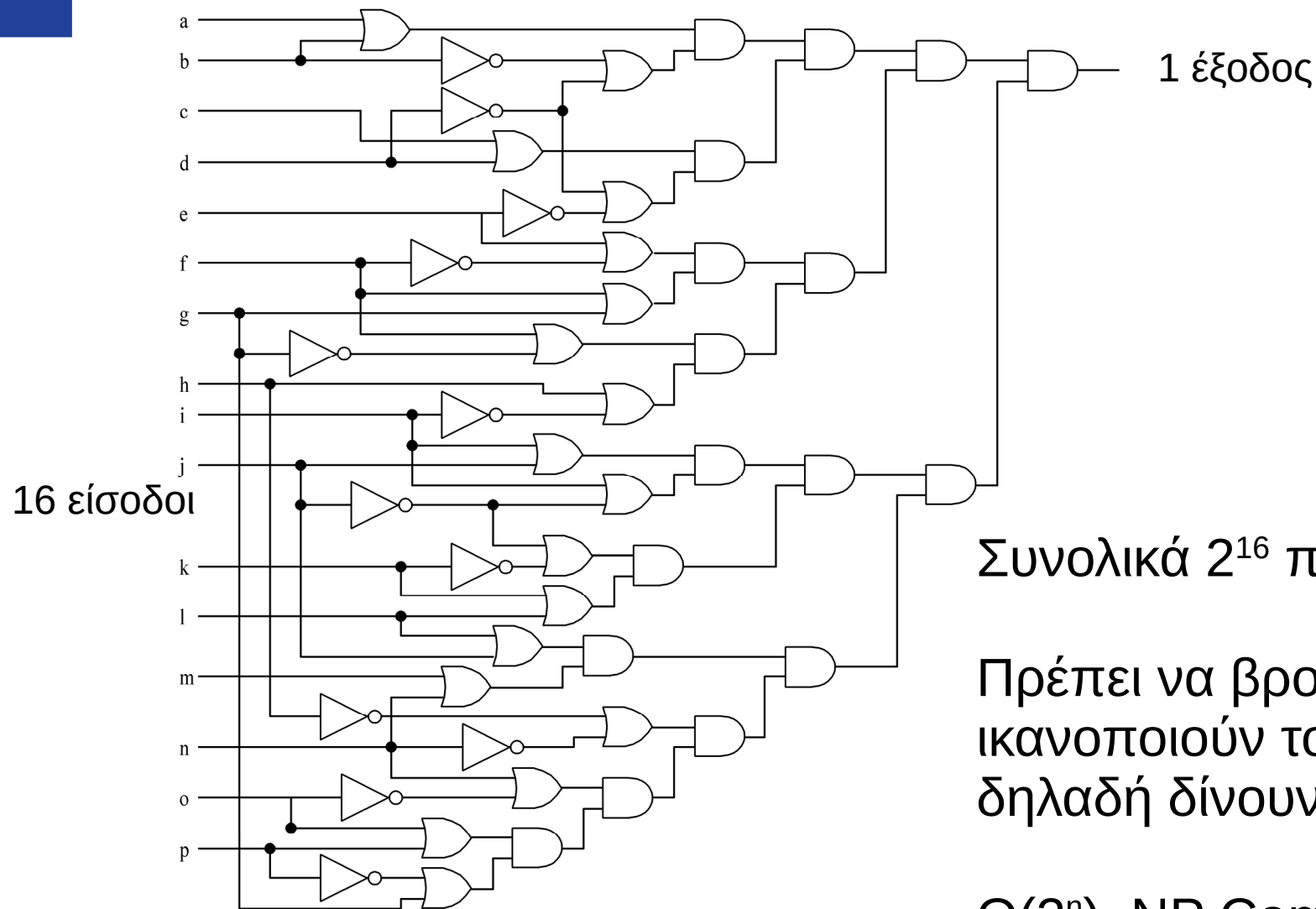
Παραλληλισμός εργασιών 1

- Ανάλυση με βάση την οργάνωση (γράφημα κλήσεων) του προγράμματος σε λειτουργίες (συναρτήσεις, κλάσεις) και τις εξαρτήσεις δεδομένων.

Πχ: ποια συνάρτηση καλεί ποιαν, με τι ορίσματα και επιστρεφόμενες τιμές.

- Έχει γενικότερη εφαρμογή αλλά δυσκολότερη παραγωγή γενικών προτύπων.
- Εστιάζουμε σε κλάσεις αλγορίθμων, όπως ανεξάρτητα πειράματα, αναδρομικούς αλγορίθμους, streaming (διοχέτευση).

Circuit satisfiability problem



Συνολικά 2^{16} πιθανές είσοδοι.

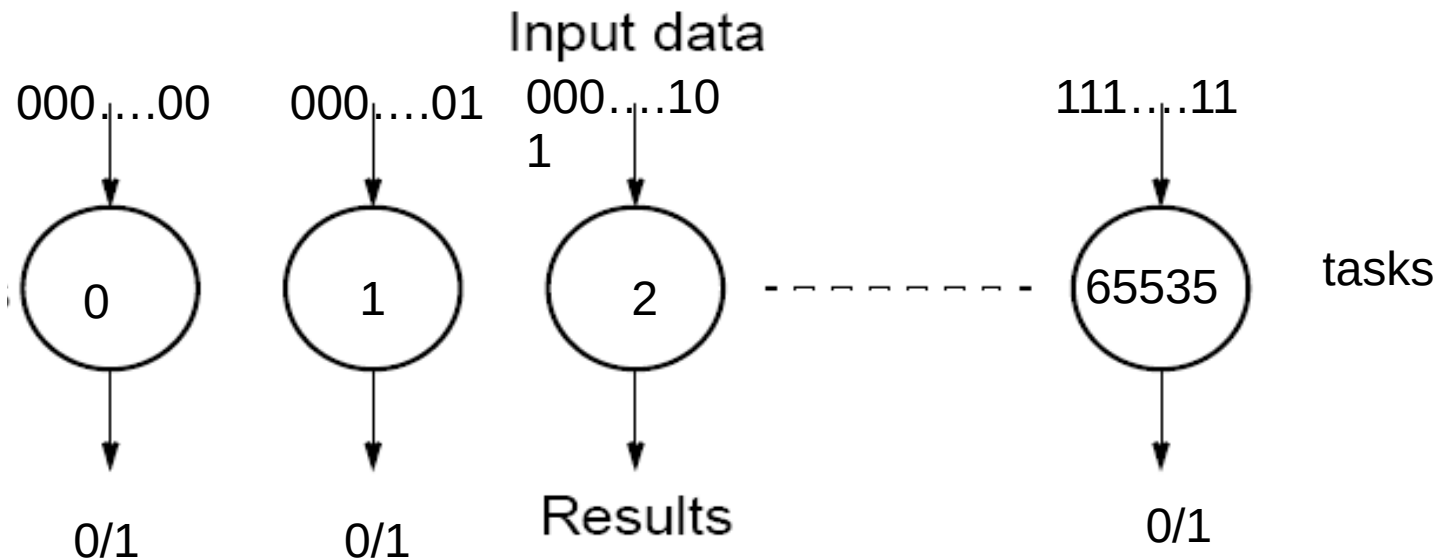
Πρέπει να βρούμε ποιες είσοδοι ικανοποιούν το λογικό κύκλωμα, δηλαδή δίνουν έξοδο 1.

$O(2^n)$, NP Complete.

https://en.wikipedia.org/wiki/Circuit_satisfiability_problem

https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

Παραλληλισμός εργασιών 2



- Πολύ απλό γράφημα με 65536 (γενικά 2^n , όπου n το πλήθος των εισόδων) ανεξάρτητες εργασίες και απλή καταγραφή του αποτελέσματος.
- Οι εργασίες μπορεί να ομαδοποιηθούν αν ο φόρτος κάθε εργασίας είναι πολύ μικρός.



Απλό παράδειγμα Πρότυπο SPMD



Απλό παράδειγμα

- Code/Sinx

Απλό παράδειγμα

- Έστω ένα άνυσμα με στοιχεία $a[i], i=0,1,\dots, N$ με αριθμούς (ή γενικότερα δεδομένα).
- Θέλουμε να παράγουμε ένα άνυσμα με στοιχεία $f(a[i]), i=0,1,\dots, N$ με την $f()$ να εξαρτάται μόνο από το $a[i]$
- Για παράδειγμα θέλουμε να υπολογίσουμε το $\sin x(a[i]), \text{sqrt}(a[i])$ ή κάποια άλλη συνάρτηση.

Ακολουθιακός αλγόριθμος

- 1 εργασία, N βήματα

```
for (i=0; i<N; i++)  
    task(i, a);
```

```
task(i, a) {  
    a[i] = f(a[i]);  
}
```

- Παραλείπουμε αρχικοποίηση και εμφάνιση αποτελεσμάτων.
- Δεν υπάρχουν εξαρτήσεις μεταξύ των N βημάτων.

Παράλληλος αλγόριθμος

- N εργασίες, 1 βήμα η κάθε μια

```
for (i=0; i<N; i++)
```

```
    create and run in parallel Task(i)
```

```
for (i=0; i<N; i++)
```

```
    wait Task(i, a) to finish;
```

```
Task(i, a) {  
    a[i] = f(a[i]);  
}
```

Το Task μπορεί να είναι νήμα, διεργασία, ότι μπορεί να έχει ανεξάρτητη ροή εκτέλεσης.

Σκελετός υλοποίησης με Νήματα

```
Shared a[N]                                // μοιραζόμενες μεταβλητές

Main {
    ...                                    // αρχικοποίηση

    for (i=0; i<N; i++) // Δημιουργία/Εκτέλεση Νημάτων
        create/start Thread (i, a);

    for (i=0; i<N; i++) // Αναμονή τερματισμού Νημάτων
        wait/join Thread (i..);

    ...                                    // αποτέλεσμα
}

Thread (i, a) {                            // Υλοποίηση Νήματος
    ...
    a[i] = f(a[i]);
    ...
}
```

Συστήματα μοιραζόμενης μνήμης

Fork-Join παραλληλισμός και

Single Program Multiple Data (SMPD) 1

- Fork-join παραλληλισμός

Το `main` δημιουργεί P εργασίες (Fork), κάθε μια με ένα ID (πχ μεταβλητή i). Οι εργασίες εκτελούνται παράλληλα, και το `main` αναμένει τον τερματισμό τους (Join).

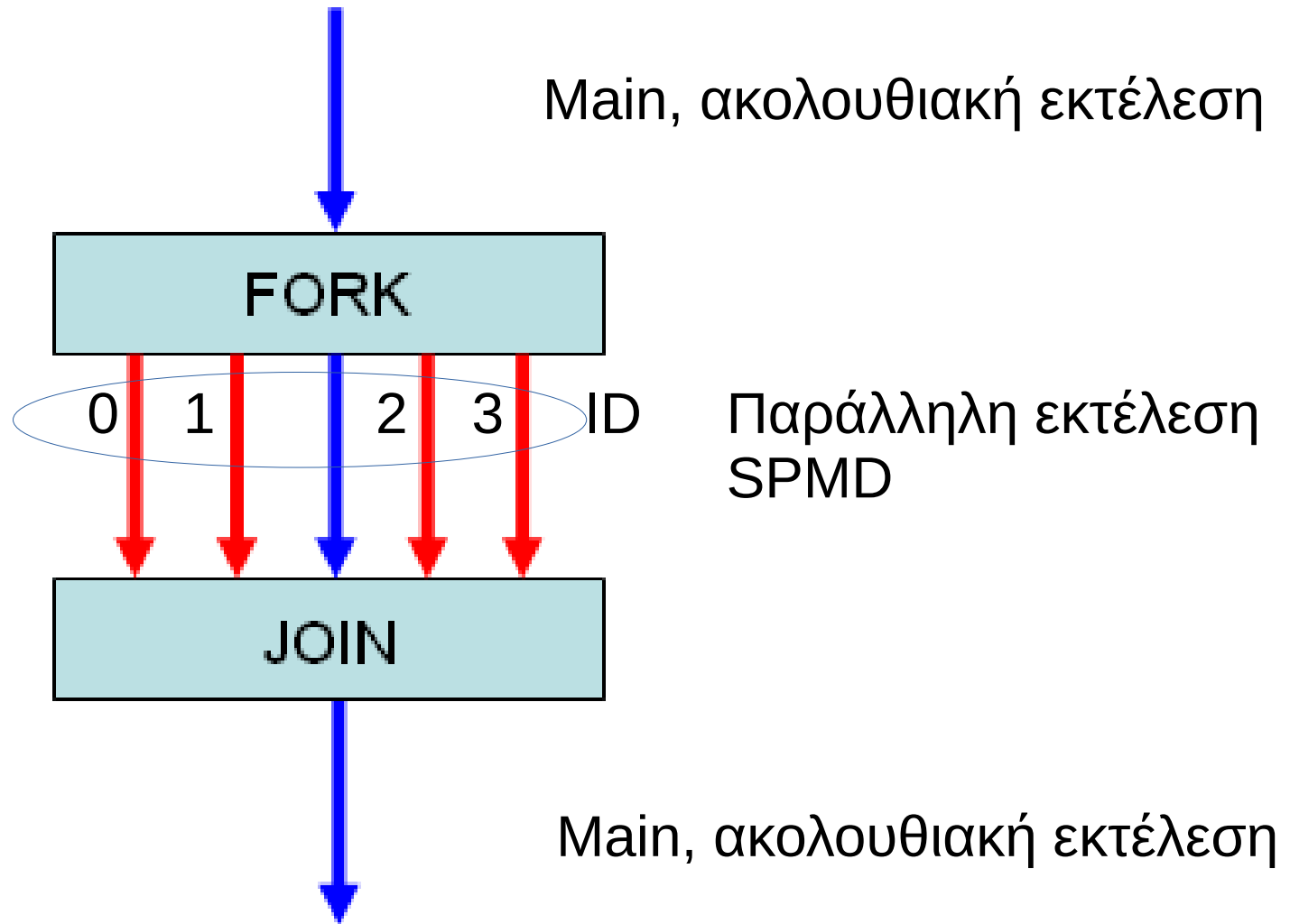
- Single Program Multiple Data

Κάθε εργασία εκτελεί το ίδιο πρόγραμμα (Single Program) αλλά σε διαφορετικά δεδομένα (Multiple Data), όπου τα δεδομένα καθορίζονται με βάση το ID του νήματος.

Συστήματα μοιραζόμενης μνήμης

Fork-Join παραλληλισμός

Single Program Multiple Data (SPMD) 2



Εναλλακτικές υλοποιήσεις

Ανάλογα με την γλώσσα / βιβλιοθήκη υπάρχουν εναλλακτικές υλοποιήσεις:

- Διεργασίες (πχ Python Multiprocessing)
- Εντολές τύπου `Parallel.FoaAll`, `Parallel.ForEach` (πχ στη C#) ή `parallel.map` (πχ Java Streams)
- Directives όπως `#pragma omp parallel for` (πχ βιβλιοθήκη OpenMP)

Υπολογισμοί SIMD: SPMD και loop unrolling

- Αρκετές CPUs (cores) διαθέτουν πολλαπλές (για παράδειγμα 2) ALUs.
- Επομένως μπορούμε να εκτελέσουμε πολλαπλά βήματα μια επανάληψης “ξεδιπλώνοντας” το βρόχο της επανάληψης (loop unroll).

Χωρίς loop unroll	Με loop unroll
<pre>for (i = 0; i<N; i++) f(i);</pre>	<pre>for (i = 0; i<N; i+=2) { f(i); f(i+1); }</pre>

Υπολογισμοί SIMD: SPMD και SIMD

- α) Οι ALUs είναι Ανυσματικές: υποστηρίζουν 4 ή 8 ίδιες πράξεις σε διαφορετικά δεδομένα (SIMD)
β) Υπάρχουν ανυσματικοί καταχωρητές που χωρούν αντίστοιχο αριθμό δεδομένων
- Η γλώσσα μηχανής διαθέτει εντολές SIMD (πχ SSE, AVX εντολές της Intel).
- Επομένως κάθε ALU μπορεί να εκτελεί ταυτόχρονα 4 ή 8 πράξεις.

Χωρίς loop SIMD	Με loop SIMD
<pre>for (i = 0; i<N; i++) f(i);</pre>	<pre>for (i = 0; i<N; i+=SIMDlen) f(i:i+SIMDlen);</pre>

Υπολογισμοί SIMD:

SPMD και loop unrolling, SIMD

- Loop unrolling: ημι-αυτόματης παραγωγή κώδικα που εφαρμόζουν αρκετοί μεταγλωττιστές ως βελτιστοποίηση (πχ gcc -O2 ή -O3).
- SIMD: Πρέπει να χρησιμοποιήσουμε ειδικές βιβλιοθήκες ή εντολές. Σε απλές περιπτώσεις μπορεί να γίνει αυτόματα.

Χωρίς loop unroll, SIMD	Με loop unroll, SIMD
<pre>for (i = 0; i<N; i++) f(i);</pre>	<pre>for (i = 0; i<N; i+=2*SIMDlen) { f(i:i+SIMDlen); f(i+SIMDlen:i+2*SIMDLEN); }</pre>

Επιταχυντές (GPUs) και SPMD

- Στους επιταχυντές το πρότυπο SPMD εφαρμόζεται περίπου όπως ο υπολογισμός loop unrolling+SIMD.
- Οι επιταχυντές διαθέτουν πολλές ALUs (πχ 2^{10}). Επιπλέον υποστηρίζουν loop unrolling+SIMD.
- Δημιουργούμε M πολύ ελαφρές εργασίες (λέγονται και αυτά νήματα) όπου το M μπορεί να πάρει σημαντικά μεγάλες τιμές (πχ $M=2^{20}$ νήματα).
- Αν $M \geq N$ (μέγεθος προβλήματος), αναθέτουμε έναν υπολογισμό σε κάθε νήμα.
- Αν $M < N$, αναθέτουμε μικρές ομάδες υπολογισμών σε κάθε νήμα.

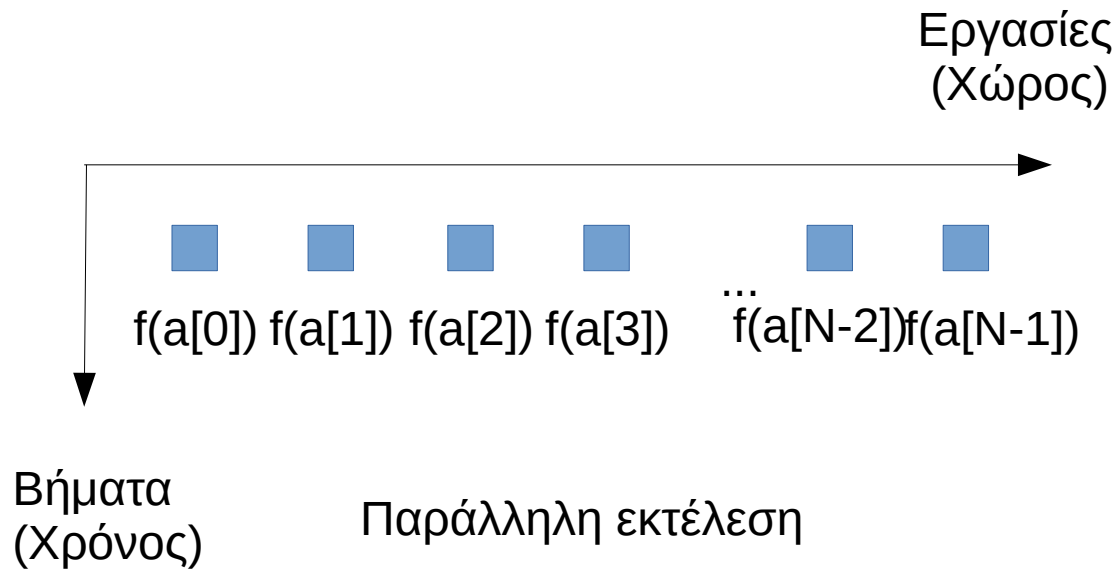
Συστήματα κατανεμημένης μνήμης και SPMD

- Στα συστήματα κατανεμημένης μνήμης το πρότυπο SPMD εφαρμόζεται ως Master Worker (θα το δούμε στην αντίστοιχη ενότητα). Οι διαφορές είναι:
- Δημιουργούμε P διεργασίες, όχι νήματα, που εκτελούνται σε διαφορετικούς υπολογιστές.
- Μία από αυτές, ο Master, έχει πρόσβαση στα δεδομένα, τα αποστέλλει στις υπόλοιπες διεργασίες Workers.
- Μετά τους υπολογισμούς, οι Workers επιστρέφουν τα αποτελέσματα των υπολογισμών στον Master.



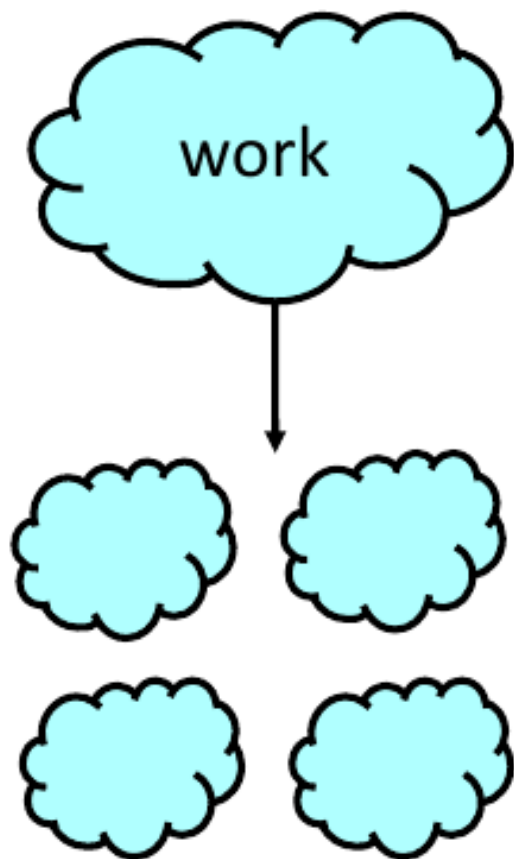
Ομαδοποίηση εργασιών

Γραφήματα εκτέλεσης

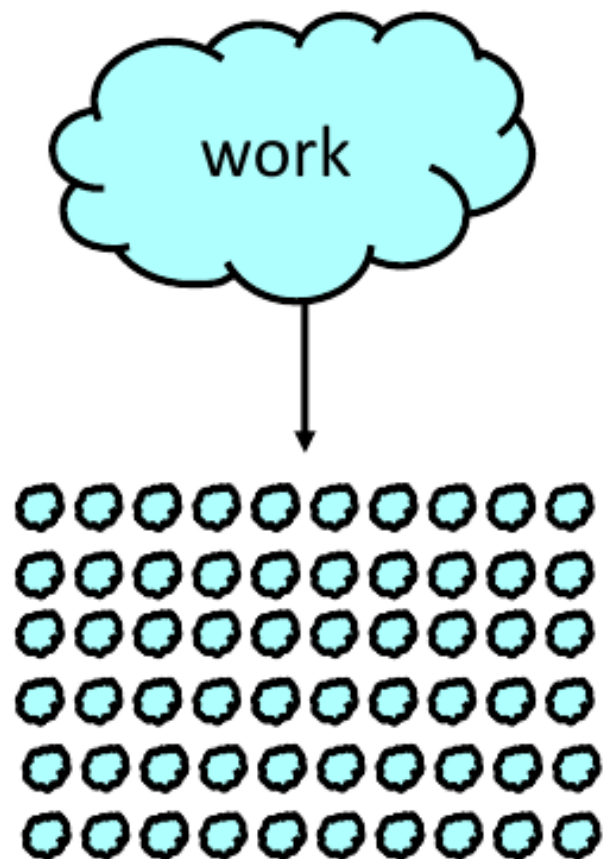


- Απλά γραφήματα εκτέλεσης: Η χρονική διαδοχή δεν καθορίζει εξάρτηση μεταξύ των εργασιών.
- Πάρα πολλές εργασίες?
- Πολύ ελαφρές εργασίες?

Ομαδοποίηση (Granularity)

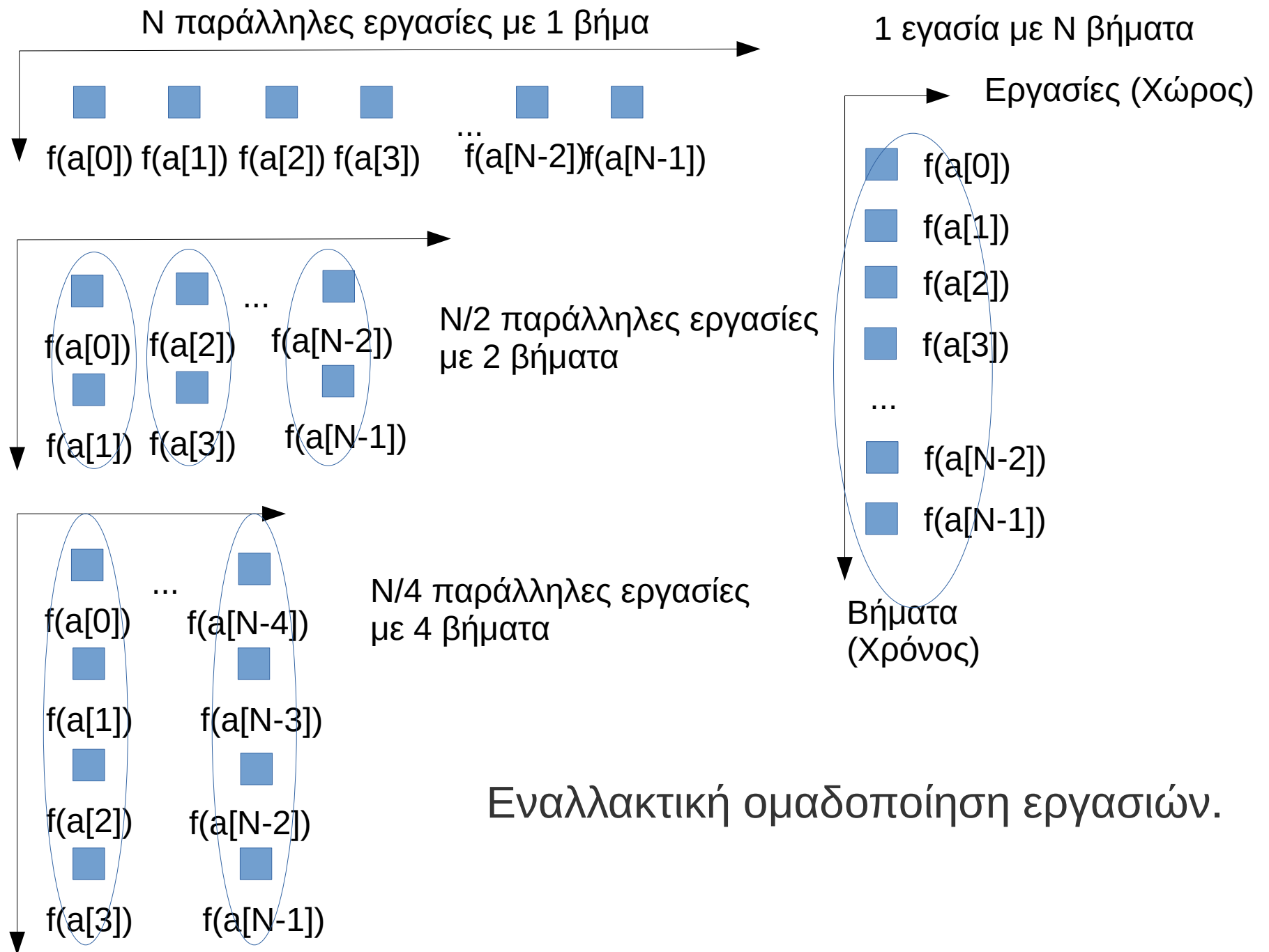


Αδρομερής (Coarse)



Λεπτομερής (Fine)

Ομαδοποίηση εργασιών 1



Ομαδοποίηση εργασιών 2

- Έστω N το μέγεθος του προβλήματος.
- Έστω P ο αριθμός των διαθέσιμων επεξεργαστών σε ένα τυπικό σύστημα μοιραζόμενης μνήμης.
- Υποθέτουμε ότι $P \ll N$.
- Θα μπορούσαμε να δημιουργήσουμε N εργασίες;
- Θα μπορούσαμε να δημιουργήσουμε $M < P$ εργασίες;

Παράμετροι ομαδοποίησης

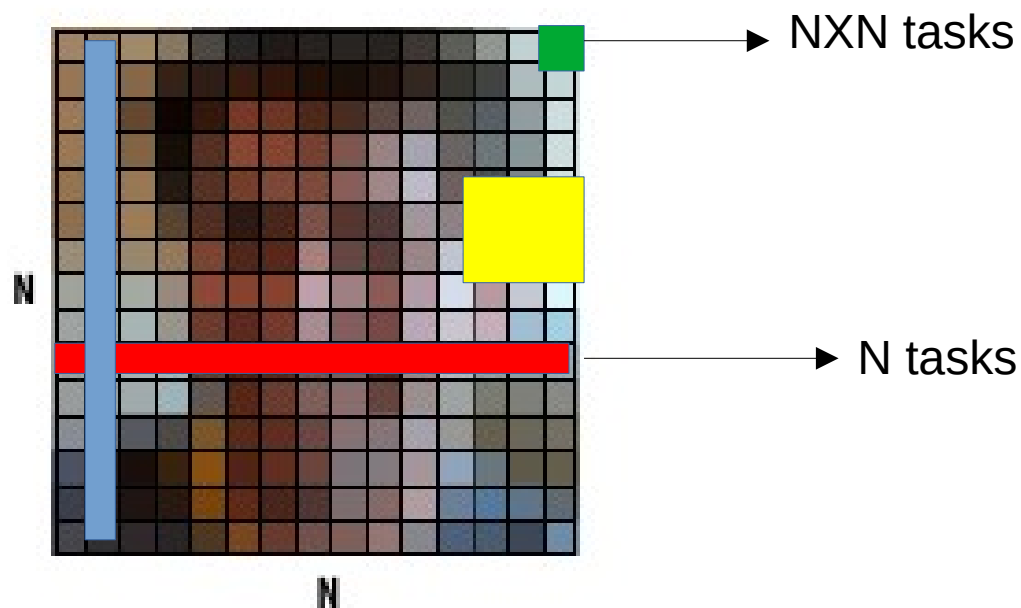
- Αριθμός εργασιών / Αριθμός επεξεργαστών

Προσπαθούμε ο αριθμός εργασιών τουλάχιστο να ισούται με τον αριθμό των διαθέσιμων επεξεργαστών (όπως τις καθορίζει το σύστημα χρόνου εκτέλεσης).

- Επιβάρυνση διαχείρισης

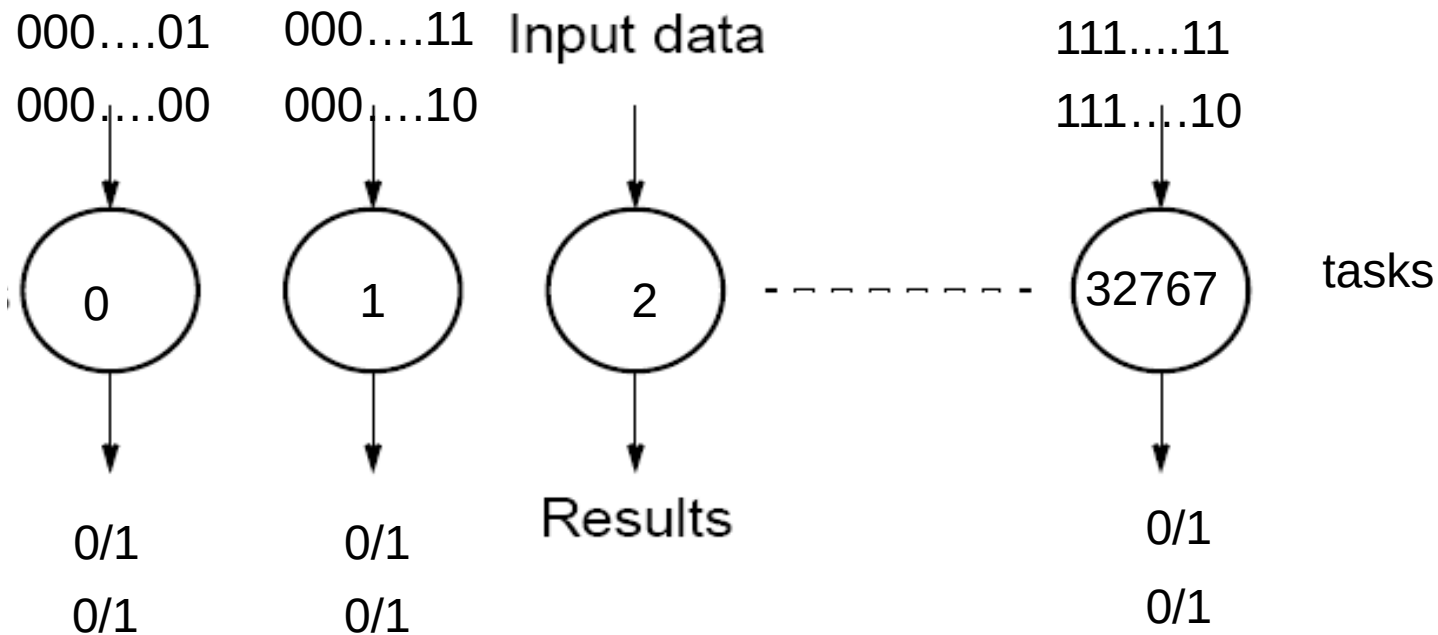
Προσπαθούμε το πλήθος των εργασιών να μην είναι υπερβολικό ώστε να μην επιβαρύνει σημαντικά την εκτέλεση (δημιουργία, κατανομή, επικοινωνία).

Παράδειγμα ομαδοποίησης 1



$N \times N$ 'ελαφρές' εργασίες ή N πιο 'βαριές' εργασίες;
Θα μπορούσαμε να έχουμε πολλές διαφορετικές
ομαδοποιήσεις.

Παράδειγμα ομαδοποίησης 2



Κάθε εργασία εκτελεί ακολουθιακά δύο υπολογισμούς και έτσι υποδιπλασιάζουμε τον αριθμό των εργασιών.

Αν κάθε εργασία εκτελεί 4096 ακολουθιακούς υπολογισμούς τότε έχουμε 16 παράλληλες εργασίες.

Βαρύτητα εργασιών: Αριθμητική πυκνότητα 1

- Αριθμητική Πυκνότητα =

Πλήθος πράξεων / πλήθος δεδομένων =

Υπολογισμός / Επικοινωνία

Ως επικοινωνία εννοούμε όχι μόνο την δικτυακή επικοινωνία αλλά και την μεταφορά δεδομένων από την μνήμη στη CPU.

Γενικά το εύρος ζώνης της επικοινωνίας είναι σημαντικά μικρότερο από την διεκπεραιωτική ικανότητα των επεξεργαστών.

Επομένως θέλουμε να έχουμε όσο το δυνατό περισσότερους υπολογισμούς ανά δεδομένο.

Βαρύτητα εργασιών: Αριθμητική πυκνότητα 2

Δύο κριτήρια:

- Ο υπολογισμός πρέπει να είναι σημαντικός πχ αρκετές επαναλήψεις.
- Αριθμός δεδομένων ανά υπολογισμό. Ιδανικά θέλουμε πολλούς υπολογισμούς ανά load, store.

Για παράδειγμα:

$a[i] = b[i] + c[i]$: 1 πράξη / 3 load, store

$a[i] = s * a[i]$: 1 πράξη / 2 load, store

$a[i] = \text{mysinx}(a[i])$: 500 πράξεις / 2 load, store

$a[i] = \text{mysqrt}(a[i])$: ... / 2 load, store

Τι συμβαίνει στα δύο προηγούμενα παραδείγματα;



Στατική κατανομή εργασιών

Υλοποίηση της ομαδοποίησης εργασιών

Κατανομή εργασιών

- Το πρόγραμμα καθορίζει την ομαδοποίηση με χρήση κάποιας μορφής κατανομής εργασιών.
- Υπάρχουν διάφορες μέθοδοι (θα δούμε αρκετές παρακάτω).
- Η απλούστερη και πιο συνηθισμένη είναι η στατική κατανομή των εργασιών.
- Αν πρόκειται για δεδομένα: η δομή δεδομένων μοιράζεται σε περίπου τμήματα στις εργασίες.
- Αν πρόκειται για πολλές μικρές εργασίες: ο βρόχος επανάληψης αναλύεται αντίστοιχα.

Στατική κατανομή εργασιών

- Έστω N το μέγεθος του προβλήματος.
- Έστω P ο αριθμός των εργασιών, $P \ll N$. Τυπικά ο αριθμός των εργασιών είναι ίσος με τον αριθμό των διαθέσιμων επεξεργαστών (ή μικρό πολλαπλάσιο)
- Οι εργασίες αριθμούνται (ID) $i = 0, 1, 2.. P-1$.
- Το πρόβλημα υποδιαιρείται σε P τμήματα, μεγέθους $block=N/P$ στοιχείων.
- Η εργασία 0 αναλαμβάνει το τμήμα 0 με στοιχεία $0...block-1$
- Η εργασία 1 αναλαμβάνει το τμήμα 1 με στοιχεία $block..2*block-1$
- ...
- Η εργασία $P-1$ αναλαμβάνει το τμήμα $P-1$ με στοιχεία $(P-1)*block..N-1$
- Το υπόλοιπο, αν υπάρχει (αν η διαίρεση N/P δεν είναι τέλεια), το αναλαμβάνει η εργασία $P-1$, ώστε τελειώνει με το στοιχείο $N-1$.

Σκελετός στατικής κατανομής 1

```
Shared a[N]                // μοιραζόμενες μεταβλητές

Main {
    ...                    // αρχικοποίηση

    for (i=0; i<P; i++)    // πλήθος νημάτων P << N
        create/start Task (i, a, P);

    for (i=0; i<P; i++)    // αναμονή τερματισμού
        wait/join Task (i..);

    ...                    // αποτέλεσμα
}
```

Σκελετός στατικής κατανομής 2

```
Task(i, a, P) {  
    block = N / P;  
    start = i * block;  
    stop = start + block;  
    if (i == P-1) stop = N;  
  
    for (j = start; j < stop; j++)  
        a[j] = f(a[j]);  
}
```

Για P εργασίες (όσοι οι επεξεργαστές): αναμενόμενος χρόνος εκτέλεσης N/P .

Σκελετός στατικής κατανομής

Εναλλακτική υλοποίηση 1

```
Shared a[N] // μοιραζόμενες μεταβλητές

Main {
    ... // αρχικοποίηση

    for (i=0; i<P; i++) { // πλήθος νημάτων P << N
        block = N / P;
        start = i * block;
        stop = start + block;
        if (i == P-1) stop = N;

        create/start Task (i, a, start, stop);
    }

    for (i=0; i<P; i++) // αναμονή τερματισμού
        wait/join Task (i..);

    ... // αποτέλεσμα
}
```

Σκελετός στατικής κατανομής Εναλλακτική υλοποίηση 2

```
Task(i, a, start, stop) {  
    for (j = start; j < stop; j++)  
        a[j] = f(a[j]);  
}
```

Για P εργασίες (όσοι οι επεξεργαστές): αναμενόμενος χρόνος εκτέλεσης N/P .

Στατική κατανομή και Ένθετες επαναλήψεις (nested for loops) 1

- Πολύ συχνά οι βρόχοι που παραλληλίζονται περιέχουν ένθετες επαναλήψεις:

Παραδείγματα:

- Εικόνες, video, γραφικά 2 ή 3 διαστάσεων
- Επαναληπτικές προσεγγίσεις --σειρά Taylor για το $\sin(x)$ ή επανάληψη Newton Raphson για το $\sqrt{x}$
- Έλεγχος για κάθε bit στο άνυσμα εισόδου του κυκλώματος --Circuit satisfiability problem

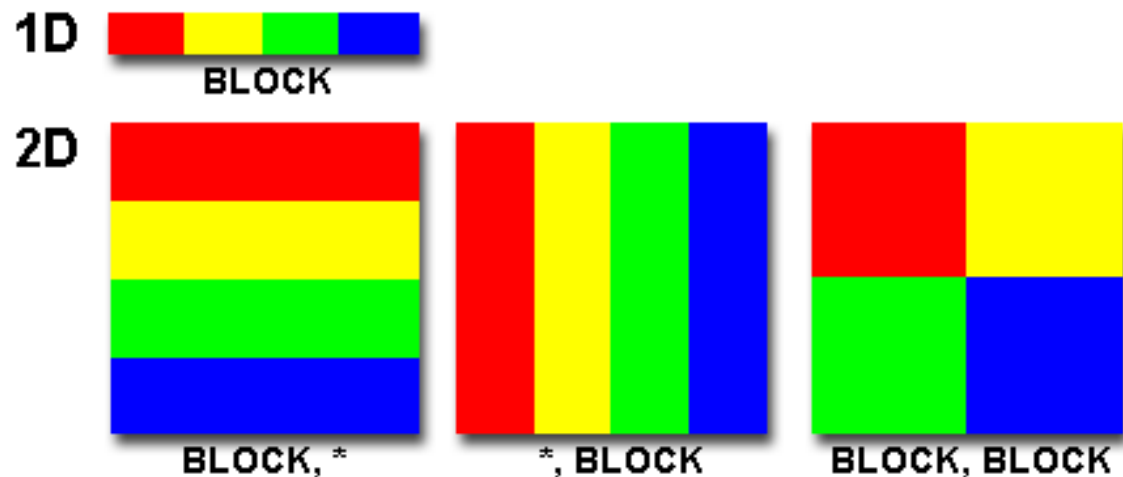
Στατική κατανομή και Ένθετες επαναλήψεις (nested for loops) 2

- Η απλούστερη προσέγγιση είναι η παραλληλοποίηση του εξωτερικού βρόχου των ένθετων επαναλήψεων.

```
for (i = 0; i < imageRows; i++) { //Στατική  
    for (j = 0; j < imageColumns; j++) //Κατανομή  
        f(imagePixel[i, j]); Task(i)  
}
```

- Θα μπορούσαμε όμως να είχαμε και στατική κατανομή ανά στήλη ή σε block.

Στατική κατανομή και Ένθετες επαναλήψεις (nested for loops) 3



- Παραδείγματα στατικής κατανομής σε μια και δύο διαστάσεις.
- Η μέθοδος κατανομής (γραμμές, στήλες, blocks) σχετίζεται με το τρόπο αποθήκευσης δεδομένων στην (κρυφή) μνήμη.